# Développement d'Applications Mobiles et IoT

## Chapitre 2 — Introduction aux architectures d'applications mobiles IoT

Pr. Abdelfattah Abassi

Faculté des Sciences et Techniques de Settat
Université Hassan I

Laboratoire LAVETE
Département de Physique Appliquée

Année universitaire 2026–2027

# Sommaire

1. Panorama des architectures mobiles

2. Architecture d'une application IoT mobile : la chaîne de valeur

# Sommaire

1. Panorama des architectures mobiles

2. Architecture d'une application IoT mobile : la chaîne de valeur

# Objectifs pédagogiques

## À l'issue de cette section, l'étudiant sera capable de :

- Distinguer les trois grandes familles d'architectures de développement mobile
- Comparer ces familles selon des critères techniques et économiques objectifs
- Justifier un choix d'architecture pour un cas d'usage IoT donné

## Contexte

En 2024, le marché mobile impose de cibler simultanément Android et iOS. La question n'est plus *« mobile ou pas »* mais *« avec quelle architecture »*.

# Les trois grandes familles

## Natif

Android (Kotlin/Java)
iOS (Swift/Obj-C)

- Un code par plateforme
- Accès total au SDK
- Performance maximale

## Hybride (WebView)

Ionic, Apache Cordova

- HTML/CSS/JS encapsulés
- Rendu via navigateur embarqué
- Plugins natifs pour le matériel

## Cross-platform natif

Flutter, React Native, Xamarin

- Un seul code source
- Compilation vers code natif
- Rendu proche du natif

## Point clé

Le cross-platform à compilation native **ne passe pas** par une WebView : le code est compilé (Flutter) ou transpilé vers des composants d'interface natifs (React Native).

# Grille de comparaison — 6 critères

| Critère | Natif | Hybride (WebView) | Cross-platform natif |
|---|---|---|---|
| Performance | ✓Excellente | ×Limitée | ✓Très bonne |
| Accès API système | ✓Total | △Via plugins | ✓Large (plugins) |
| Coût développement | ×Élevé (x2 code) | ✓Faible | ✓Faible |
| Expérience utilisateur | ✓Native pure | ×Approximative | ✓Quasi native |
| Maintenabilité | ×2 bases de code | ✓Unique | ✓Unique |
| Communauté | ✓Très large | △En déclin | ✓Large et active |

# Cas d'usage IoT : quelle architecture ?

## Application de supervision IoT — besoins types

- Connexion BLE/Wi-Fi à des objets connectés
- Affichage de données temps réel (graphiques, jauges)
- Notifications push, accès capteurs du téléphone
- Déploiement rapide sur Android **et** iOS

## Conclusion pédagogique

Pour une application IoT, le **cross-platform** (Flutter en tête) s'impose : unicité du code, accès suffisant aux API BLE/réseau via plugins, coût maîtrisé, et time-to-market réduit — des contraintes réelles de la supervision IoT.

## Nuance

Le natif reste préférable pour un accès matériel très pointu (drivers BLE propriétaires, traitement temps réel critique) ou une seule plateforme cible.

# Sommaire

1 Panorama des architectures mobiles

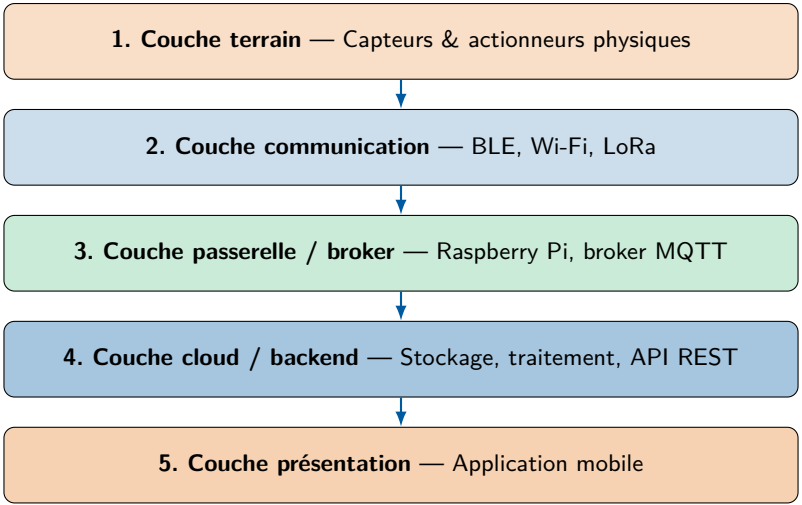
2 Architecture d'une application IoT mobile : la chaîne de valeur

# Objectifs pédagogiques

## À l'issue de cette section, l'étudiant sera capable de :

- Modéliser un système IoT mobile en 5 couches
- Identifier technologies et flux de données propres à chaque couche
- Cartographier un système IoT réel selon ce modèle

# Modèle en 5 couches



# Couche 1 — Terrain

## Composants

- Capteurs : température, humidité, mouvement, luminosité...
- Actionneurs : relais, moteurs, LED, vannes...
- Microcontrôleurs : ESP32, Arduino, STM32

## Flux & interfaces

- Mesures physiques → signaux numériques
- Interface vers la couche 2 : radio intégrée (BLE, Wi-Fi) ou module externe (LoRa)

## Contrainte de conception

Faible puissance de calcul et énergie limitée ⇒ protocoles légers exigés dès cette couche.

# Couche 2 — Communication sans fil

| | **BLE** | **Wi-Fi** | **LoRa** |
|---|---|---|---|
| Portée | ∼10–30 m | ∼50 m | ∼2–15 km |
| Débit | Faible | Élevé | Très faible |
| Consommation | Très faible | Élevée | Très faible |
| Usage IoT typique | Wearables, capteurs proches | Passerelles domestiques | IoT longue portée |

## Interface avec les couches adjacentes

Reçoit les données brutes de la couche terrain, les transmet à la passerelle (couche 3) selon un protocole applicatif (souvent MQTT au-dessus du transport).

# Couche 3 — Passerelle & Broker (1/3) : L'Analogie

## C'est quoi un Broker ? (L'analogie du réseau social / de la poste)

Imaginez un **bureau de poste** ou un **panneau d'affichage général** :

- Les capteurs ne se parlent **jamais** directement entre eux.
- Un capteur dépose un message adressé à une catégorie (un **Topic**).
- Le Broker distribue le message à toutes les personnes inscrites à ce topic.

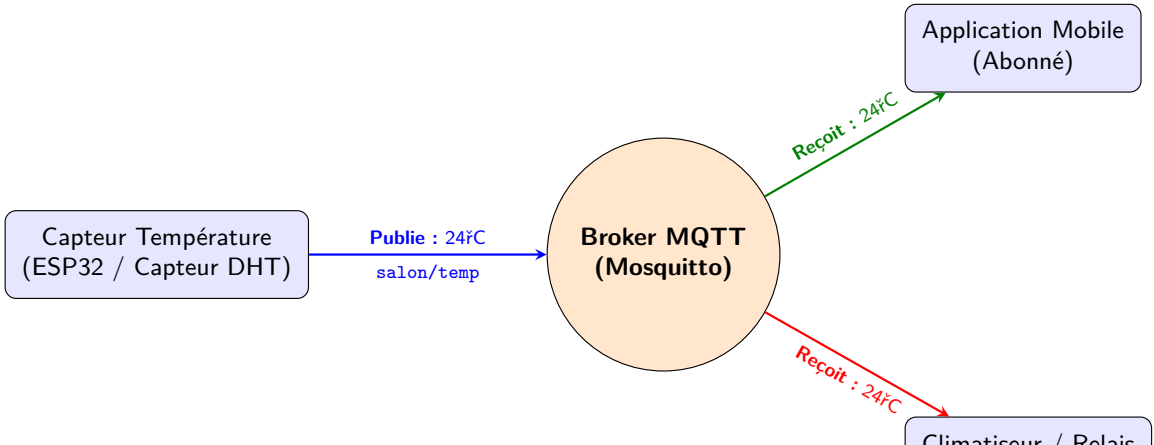
## Exemple de la vie courante

- **Émetteur (Publisher) :** Un journal de météo publie un bulletin sur le sujet ń Température ż.
- **Le Broker :** Le kiosque à journaux qui centralise les parutions.
- **Régulateur/Abonné (Subscriber) :** Vous achetez ce journal car le sujet ń Température ż vous intéresse.

# Couche 3 — Passerelle & Broker (2/3) : Fonctionnement MQTT

## Le principe : Publish / Subscribe (Pub/Sub)

Les objets échange des données via des canaux nommés **Topics** (ex : `maison/salon/temperature`).



# Couche 3 — Passerelle & Broker (3/3) : Cas Pratique

## Exemple concret : Gestion de température intelligente

1. **Acquisition (Capteur Bluetooth) :** Un petit capteur mesure 26 °C dans une chambre. Il n'a pas accès à Internet, il utilise le **Bluetooth (BLE)**.
2. **Traduction (Passerelle / Gateway) :** Un **Raspberry Pi** situé dans la maison capte le signal Bluetooth et le convertit en paquet **Wi-Fi / IP**.
3. **Acheminement (Broker MQTT) :** Le logiciel **Mosquitto** (installé sur le Raspberry Pi ou dans le Cloud) reçoit l'information sur le topic `maison/chambre/temp`.
4. **Action :** Le smartphone de l'utilisateur reçoit la mise à jour et affiche la courbe sur son écran.

## Outils et technologies à retenir pour un projet d'études

- **Matériel Passerelle :** Raspberry Pi 4 / ESP32 Gateway.
- **Brokers populaires :** Eclipse Mosquitto (léger), EMQX (cloud/industriel), HiveMQ.

# Couche 4 — Cloud / Backend

## Fonctions

- Stockage persistant (bases de données temps réel ou relationnelles)
- Traitement : agrégation, règles métier, alertes, historisation
- Exposition d'une **API REST** (ou WebSocket) consommée par l'application mobile

## Exemples de technologies

Firebase, AWS IoT Core, backends FastAPI/Node.js + base de données (SQL ou NoSQL)

# Couche 5 — Présentation (application mobile)

## Rôle

- Consomme l'API REST (ou WebSocket/MQTT direct) de la couche 4
- Affiche les données en temps réel : jauges, graphiques, historiques
- Permet le pilotage : envoi de commandes vers les actionneurs (retour vers la couche terrain)

## Boucle complète

La couche présentation peut renvoyer une commande qui redescend les couches 4 $\rightarrow$ 3 $\rightarrow$ 2 $\rightarrow$ 1 : le modèle est **bidirectionnel**.

# Exercice de cartographie

## Application

Pour un système de **maison intelligente** (thème fil rouge du module), identifier pour chaque couche :

1. Le ou les composants concrets utilisés
2. La technologie de communication choisie
3. Le format des données échangées

## À retenir

Ce modèle en 5 couches sera réutilisé tout au long du module comme grille de lecture de tout système IoT rencontré.

## Prochain chapitre

Installation de l'environnement et création d'une première application Flutter

Questions ?